6. Разработка Web UI. JavaScript

Цель работы

Научиться обращаться к Web API из обозревателя и отображать полученные данные. Работа завершает семестровый проект: интерфейс строится поверх API, созданного в предыдущих работах.

Материалы для подготовки

- Запросы средствами Fetch: https://developer.mozilla.org/docs/Web/API/Fetch_API/Using_Fetch
- Объектная модель документа: https://developer.mozilla.org/docs/Web/API/Document_Object_Model
- Настройка CORS: https://learn.microsoft.com/aspnet/core/security/cors
- Статические файлы в ASP.NET Core: https://metanit.com/sharp/aspnet5/25.2.php
- Методы HTTP: https://developer.mozilla.org/docs/Web/HTTP/Methods

Теоретические сведения

Из чего состоит интерфейс

Страница получает данные от API теми же запросами, что отправлялись из Swagger в предыдущих работах. Разница в том, что запрос выполняет код на JavaScript, а результат превращается в элементы страницы.

Примеры этого раздела построены на другой предметной области: книгах и издательствах. Переложить их на автомобили и дилеров предстоит самостоятельно.

Запрос к API

Запрос выполняется функцией `fetch`. Она возвращает обещание, поэтому результат дожидаются оператором `await`.

```
async function loadBooks() {
    const response = await fetch("http://localhost:5000/api/books");
    if (!response.ok) {
        throw new Error("Запрос завершился с кодом " + response.status);
    }
    return await response.json();
}
```

Свойство `ok` истинно для кодов диапазона `2xx`. Ответ `404` исключения не вызывает, поэтому код состояния проверяется явно.

Построение разметки

Каждая запись отображается отдельным блоком. Блоки создаются в цикле по полученному массиву и добавляются в контейнер на странице.

```
const container = document.querySelector("#books");
container.innerHTML = "";

for (const book of books) {
```

```
    const card = document.createElement("div");
    card.className = "card";
    card.append(createTitle(book), createButtons(book));
    container.append(card);
}
```

Подстановка данных через `innerHTML` со склейкой строк опасна: значение поля попадёт на страницу как разметка. Текст добавляется через `textContent` либо через создание узлов.

Изменение и удаление

Кнопки внутри блока отправляют запросы теми же методами, что проверялись в четвёртой работе.

```
await fetch(`http://localhost:5000/api/books/${id}`, {
    method: "DELETE",
});
```

При замене записи тело запроса сериализуется в JSON, а заголовок сообщает серверу тип содержимого.

```
await fetch(`http://localhost:5000/api/books/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(book),
});
```

После успешного изменения список перечитывается заново либо изменённый блок обновляется на месте.

Обращение к чужому источнику

Страница, открытая с одного адреса, по умолчанию не может обратиться к API на другом адресе: обозреватель заблокирует ответ. Различие в порте считается различием адреса, поэтому страница с `localhost:3000` и API на `localhost:5000` это разные источники.

Разрешение выдаётся на стороне API настройкой CORS. Обозреватель предварительно отправляет служебный запрос методом `OPTIONS` и по его ответу решает, пропускать ли основной.

Второй способ обойтись без настройки состоит в том, чтобы отдавать страницу тем же приложением, что и API, из каталога статических файлов. Тогда источник один и запрет не действует.

> Ошибка в консоли обозревателя со словом CORS означает, что запрос дошёл до сервера и вернулся, но ответ заблокирован обозревателем. Искать причину в коде страницы бесполезно, настраивается сторона API.

Задания

1. Создайте страницу и подключите к ней код на JavaScript.
2. Реализуйте получение всех автомобилей и вывод каждого отдельным блоком.
3. Реализуйте поиск автомобиля по идентификатору с отображением результата.
4. Обработайте случай, когда автомобиль с указанным идентификатором не найден.
5. Добавьте в блок каждого автомобиля кнопку изменения, отправляющую запрос методом `PUT`.
6. Добавьте кнопку удаления, отправляющую запрос методом `DELETE`, и обновляйте список после удаления.
7. Настройте доступ страницы к API одним из двух описанных способов и укажите в отчёте, каким именно.
8. Оформите отчёт по требованиям из общей части методических указаний.

Дополнительные задания

Задания этого раздела необязательны и учитываются при оценивании.

1. Добавьте к каждому автомобилю изображение, получаемое автоматически, а не подставленное вручную.
2. Обеспечьте пригодность интерфейса для мобильных устройств.
3. Постройте интерфейс средствами какой-либо библиотеки или фреймворка, например ReactJS.

Критерии приёмки

- Данные отображаются отдельным блоком на каждую запись.
- Поиск по идентификатору работает, отсутствие записи обрабатывается и сообщается пользователю.
- Изменение выполняется методом `PUT`, удаление методом `DELETE`.
- Список приводится в соответствие с состоянием базы после изменения и удаления.
- Данные попадают на страницу как текст, а не как разметка.
- Код состояния ответа проверяется, ошибка запроса не остаётся незамеченной.
- Студент объясняет, почему обозреватель ограничивает обращение к чужому источнику и как это решено в его работе.

Контрольные вопросы

1. Что возвращает `fetch` и зачем нужен `await`?
2. Почему ответ `404` не приводит к исключению и как его обнаружить?
3. Чем `textContent` отличается от `innerHTML` и почему второе опасно для данных из API?
4. Какие заголовки нужны запросу с телом в формате JSON?
5. Что такое источник и почему `localhost:3000` и `localhost:5000` считаются разными?

6. Какой запрос обозреватель отправляет перед основным и что он проверяет?
7. Каким способом вы решили вопрос доступа к API и какая альтернатива у него была?